

EasyXT

策略开发指南 · 从模板到实盘

作者	王者quant
版本	V1.0
受众	进阶开发者
更新时间	2026 年 7 月

目录

一、策略开发基础

1.1 策略文件结构

1.2 最小策略模板

1.3 项目策略规范

二、数据获取

2.1 DuckDB 优先策略

2.2 QMT 降级模式

2.3 分红/财务数据

三、信号生成

3.1 单因子信号

3.2 多因子排名融合

3.3 技术指标信号

四、下单执行

4.1 限价单与市价单

4.2 价格取整

4.3 卖出数量安全处理

五、实盘部署

5.1 dry_run 安全模式

5.2 .env 配置管理

5.3 定时调度

六、多策略调度

6.1 StrategyCoordinator

6.2 VirtualBookkeeper

6.3 冲突处理

七、内置策略解析

7.1 红利低波

7.2 可转债双低/三低

7.3 ETF 趋势

策略开发基础

1.1 策略文件结构

一个标准的 EasyXT 策略由两个文件组成：

文件	职责	示例
策略核心	策略类，包含信号生成、风控逻辑	<code>dividend_lowvol.py</code>
运行入口	参数配置、API 初始化、环境读取	<code>run_dividend_lowvol.py</code>

分离的好处：核心策略可以独立测试和回测，运行入口专注于参数和部署。同一个策略核心可以被命令行、GUI、调度器三种方式调用。

1.2 最小策略模板

核心策略类的最小实现：

```

import pandas as pd
import numpy as np
from typing import Tuple, Dict, List

class MyStrategy:
    """我的策略"""

    def __init__(self, api=None, account_id=None, **kwargs):
        self.api = api
        self.account_id = account_id
        self.buy_n = kwargs.get('buy_n', 10)
        self.positions = None

    def generate_signals(self) -> Tuple[pd.DataFrame, pd.DataFrame]:
        """生成买卖信号。

        Returns:
            buy_list: DataFrame with code, price, ...
            sell_list: DataFrame with code, price, reason ...
        """
        # === 1. 获取数据 ===
        # === 2. 计算信号 ===
        # === 3. 返回买卖列表 ===
        return pd.DataFrame(), pd.DataFrame()

    def run(self, dry_run: bool = True):
        """运行策略"""
        buy_list, sell_list = self.generate_signals()

        # 打印信号
        print(f"买入: {len(buy_list)} 只")
        print(f"卖出: {len(sell_list)} 只")

        if not dry_run:
            self._execute_orders(buy_list, sell_list)

    def _get_positions(self) -> pd.DataFrame:
        """获取当前持仓"""
        if not self.account_id or not self.api:
            return pd.DataFrame()
        return self.api.trade.get_positions(self.account_id)

    def _execute_orders(self, buy_list, sell_list):
        """执行买卖"""
        for _, row in sell_list.iterrows():
            self.api.trade.sell(self.account_id, row['code'],
                                volume=100, price=row['price'], price_type='limit')
        for _, row in buy_list.iterrows():

```

```
self.api.trade.buy(self.account_id, row['code'],
                   volume=100, price=row['price'], price_type='limit')
```

运行入口:

```
import sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.dirname(__file__))))
from easy_xt import get_api
from strategies.quant_strategies.my_strategy import MyStrategy

# 读取 .env
def _env(key, default=''):
    with open('.env', encoding='utf-8') as f:
        for line in f:
            if line.startswith(f'{key}='):
                return line.split('=', 1)[1].strip()
    return default

PARAMS = {'buy_n': 10}
dry_run = '--trade' not in sys.argv

api = get_api()
api.init_data()
if not dry_run:
    api.init_trade(_env('QMT_DATA_DIR'), 'my_session')
    api.add_account(_env('QMT_ACCOUNT_ID'), 'STOCK')

strategy = MyStrategy(api=api, account_id=_env('QMT_ACCOUNT_ID'), **PARAMS)
strategy.run(dry_run=dry_run)
```

1.3 项目策略规范

规范	说明
文件命名	策略核心： <code>strategy_name.py</code> ； 运行入口： <code>run_strategy_name.py</code>
类命名	驼峰命名，英文描述，如 <code>DividendLowVolStrategy</code>
参数管理	运行入口的 <code>PARAMS</code> 字典 + 环境变量 <code>.env</code>
数据来源	优先 DuckDB，降级 QMT API
日志	使用 <code>logging.getLogger(__name__)</code> ， 关键步骤打印 <code>logger.info()</code>
安全模式	<code>--trade</code> 参数控制是否实际下单

二 数据获取

2.1 DuckDB 优先策略

策略中的数据获取应采用"DuckDB 优先、QMT 降级"的模式：

```
def get_stock_prices() -> Dict[str, float]:
    """获取全市场最新收盘价，优先 DuckDB，降级 QMT"""
    # 方式1: DuckDB (毫秒级)
    try:
        import duckdb
        con = duckdb.connect('D:/StockData/stock_data.ddb', read_only=True)
        latest = con.execute("SELECT MAX(date) FROM stock_daily").fetchone()[0]
        rows = con.execute(f"""
            SELECT stock_code, close FROM stock_daily
            WHERE date = '{latest}' AND close > 0
        """).fetchall()
        con.close()
        if rows:
            return {r[0]: r[1] for r in rows}
    except Exception:
        pass

    # 方式2: QMT API 降级 (秒级)
    try:
        from easy_xt import get_api
        api = get_api()
        api.init_data()
        df = api.get_current_price(api.get_stock_list())
        return dict(zip(df['code'], df['price']))
    except Exception:
        return {}
```

2.2 QMT 降级模式

EasyXT 框架自动处理数据源降级。在策略中直接使用 `api.get_price()`，框架会自动选择最佳数据源。

性能对比： DuckDB 全市场收盘价查询 ~50ms，QMT API 同样查询 ~3-5 秒。高频策略应尽量用 DuckDB 预下载数据。

2.3 分红/财务数据

分红数据通过 DuckDB `dividend_data` 表获取，无数据时降级到 QMT `get_divid_factors()`：

```
def get_dividend_data() -> pd.DataFrame:
    try: # DuckDB
        import duckdb
        con = duckdb.connect('D:/StockData/stock_data.ddb', read_only=True)
        df = con.execute("""SELECT ts_code as code, cash_div, ex_date
            FROM dividend_data WHERE cash_div > 0 ORDER BY ex_date DESC""").fetchdf()
        con.close()
        if not df.empty: return df
    except: pass

    # 降级 QMT
    from xtquant import xtdata
    results = []
    for code in stock_list:
        data = xtdata.get_divid_factors(code, '20150101', '20991231')
        if data is not None and not data.empty:
            last = data.sort_values('time').iloc[-1]
            if float(last['interest']) > 0:
                results.append({'code': code, 'cash_div': float(last['interest'])})
    return pd.DataFrame(results)
```

三 信号生成

3.1 单因子信号

最简单的信号生成——按某个指标排名选股：

```
# 按股息率降序选前 N 只
pool['yield_rank'] = pool['dividend_yield'].rank(ascending=False)
buy_list = pool.nsmallest(buy_n, 'yield_rank')
```

3.2 多因子排名融合

对每个因子分别排名，加权求和得到综合得分。推荐使用排名法而非原始值法——排名法量纲无关、极端值鲁棒：

```
# 因子1: 股息率（越高越好 → 降序排名）
pool['yield_rank'] = pool['dividend_yield'].rank(ascending=False)

# 因子2: 波动率（越低越好 → 升序排名）
pool['vol_rank'] = pool['volatility'].rank(ascending=True)

# 因子3: 市值（越小越好 → 升序排名）
pool['size_rank'] = pool['market_cap'].rank(ascending=True)

# 综合得分（权重可调）
pool['score'] = (pool['yield_rank'] * 1.0 +
                 pool['vol_rank'] * 0.5 +
                 pool['size_rank'] * 0.3)

# 得分最低的 N 只入选
selected = pool.nsmallest(buy_n, 'score')
```

为什么不直接加权原始值？ 股息率 2%-15%，波动率 15%-50%，直接用原始值加权会把波动率的影响放大 10 倍。排名打分是量贩策略的通用做法。

3.3 技术指标信号

结合 MA、RSI、MACD 等技术指标生成交易信号：

```
def generate_ma_signal(code, price_map):
    closes = get_historical_closes(code, period=60)
    if len(closes) < 20: return None

    ma5 = np.mean(closes[-5:])
    ma20 = np.mean(closes[-20:])
    prev_ma5 = np.mean(closes[-6:-1])
    prev_ma20 = np.mean(closes[-21:-1])

    # 金叉买入
    if prev_ma5 <= prev_ma20 and ma5 > ma20:
        return 'buy'
    # 死叉卖出
    elif prev_ma5 >= prev_ma20 and ma5 < ma20:
        return 'sell'
    return None
```

四 下单执行

4.1 限价单与市价单

类型	price_type	price	说明
限价单	'limit'	>0 的具体价格	按指定价格下单，可能不成交
市价单	'market'	0	以对手价成交，保证成交但不保证价格

```
# 限价 12.50 买入
api.trade.buy(account_id, '000001.SZ', volume=100, price=12.50, price_type='limit')

# 市价卖出
api.trade.sell(account_id, '000001.SZ', volume=100, price=0, price_type='market')
```

4.2 价格取整

不同品种有不同的最小价格变动单位，必须取整否则废单：

```
def round_price(code, price):
    """按交易品种取整到交易所最小价差"""
    code_str = str(code).replace('.SH','').replace('.SZ','')
    if code_str.startswith(('1','12')):    # 可转债 → 0.001
        return round(price, 3)
    elif code_str.startswith(('5','15','16','588')): # ETF → 0.001
        return round(price, 3)
    else:                                # 股票 → 0.01
        return round(price, 2)
```

4.3 卖出数量安全处理

卖出时取 `enable_amount`（可用数量）而非 `volume`（总持仓），避免挂单中的持仓被重复卖出导致废单：

```
def get_safe_sell_volume(positions, code, default=100):
    """获取安全的卖出数量"""
    match = positions[positions['code'].str.contains(code.replace('.SH','').replace('.SZ',''))]
    if match.empty: return default
    row = match.iloc[0]
    enable = int(row.get('enable_amount', 0) or row.get('can_use_volume', 0) or 0)
    if enable > 0: return enable
    volume = int(row.get('volume', 0) or 0)
    return max(volume, default) if volume > 0 else default
```

五 实盘部署

5.1 dry_run 安全模式

所有策略必须支持 dry_run 模式——只输出信号，不实际下单：

```
# 安全模式（不下单）
python strategies/quant_strategies/run_dividend_lowvol.py

# 实盘模式
python strategies/quant_strategies/run_dividend_lowvol.py --trade
```

铁律：实盘前必须先用 dry_run 模式跑至少 1-2 天，确认信号逻辑、资金计算、买卖数量都正确。

5.2 .env 配置管理

策略运行入口从 `.env` 读取敏感配置，不硬编码：

```
def _env(key, default=''):
    """从 .env 读取配置"""
    p = os.path.join(PROJECT_ROOT, '.env')
    try:
        with open(p, encoding='utf-8') as f:
            for line in f:
                if line.startswith(f'{key}='):
                    return line.split('=', 1)[1].strip()
    except: pass
    return default

QMT_PATH = _env('QMT_DATA_DIR', '')
ACCOUNT_ID = _env('QMT_ACCOUNT_ID', '')
```

5.3 定时调度

生产环境建议用 Windows 任务计划程序或 cron 定时触发：

```
# Windows 任务计划程序示例
# 程序: python.exe
# 参数: D:/EasyXT/strategies/quant_strategies/run_dividend_lowvol.py --trade
# 触发器: 每月最后一个交易日 14:50

# Linux cron 示例
# 50 14 * * 1-5 cd /opt/EasyXT && python strategies/quant_strategies/run_dividend_lowvol.py
```

也可以使用项目内置的调度器运行多策略:

```
python strategies/scheduler/strategy_scheduler.py
```

六 多策略调度

6.1 StrategyCoordinator

多策略协调器负责：统一初始化交易连接、按比例分配资金、去重合并订单、先卖后买执行。



关键能力：

- **订单去重**：同一只证券只能被一个策略交易，避免买卖冲突
- **资金分配**：按策略的 `target_allocation` 比例分配可用资金
- **虚拟持仓**：各策略独立追踪持仓，不与 QMT 真实持仓混淆

6.2 VirtualBookkeeper

虚拟账簿记录每个策略的独立持仓，策略调用 `get_positions()` 时返回虚拟持仓而非 QMT 真实持仓：

```
# 协调器注入虚拟持仓
strategy.positions = virtual_bookkeeper.get_positions(strategy_name)

# 策略内 get_positions() 优先用虚拟持仓
def _get_positions(self):
    if self.positions is not None:      # 被注入的虚拟持仓
        return self.positions
    return self.api.trade.get_positions(self.account_id) # 回退 QMT
```


6.3 冲突处理

规则	说明
<code>allow_same_security: false</code>	同一证券只被一个策略持有
<code>max_total_capital</code>	所有策略合计资金上限
<code>reserve_ratio: 0.1</code>	预留 10% 现金不动用

七 内置策略解析

剖析 EasyXT 内置策略的实现，帮助理解策略开发模式。

7.1 红利低波策略

环节	实现
数据	DuckDB stock_daily（股价）+ dividend_data（分红），降级 QMT get_divid_factors
股息率	每股最新分红 / 当前股价 × 100%
波动率	60 日对数收益率年化标准差
门槛	股息率 ≥ 2%，波动率 ≤ 50%
打分	股息率排名 × 1.0 + 波动率排名 × 0.5
买入	得分最低的前 20 只，不重复当前持仓
卖出	持仓跌出前 40 名时清仓

7.2 可转债双低/三低策略

策略	公式	逻辑
双低	转债价格 + 转股溢价率	越小越好，代表偏债（保护）和偏股（进攻）的平衡
三低	双低值 + 剩余规模惩罚	在双低基础上排除大规模债

均线多头排列过滤器：MA5 > MA10 > MA20 > MA30，每满足一层 +25 分，低于 50 分不买。

7.3 ETF 趋势策略

环节	实现
选池	主要 ETF（沪深300、创业板、科创50、恒生等）
趋势判断	收盘价在 MA20 之上为上升趋势
排序	按近期涨幅排名，买入最强的前 N 只
止损	跌破 MA20 或回撤超过 8% 清仓

学习路径建议：先读懂红利低波（逻辑最清晰），再看 ETF 趋势（技术指标入门），最后研究可转债双低（多因子融合）。三个策略覆盖了基本面因子、技术因子、多品种三大方向。

— 文档结束 —

EasyXT 量化交易框架 · 策略开发指南

王者quant · 2026 年 7 月